

Отчёт по лабораторной работе №2

Система контроля версий git, управление версиями

Магос Иоаннис

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Вывод	11
4	Контрольные вопросы	12

Список иллюстраций

2.1	Вывод команды git	6
2.2	Мая новая учётная запись	7
2.3	Параметры репозитория	7
2.4	rsa-4096	7
2.5	ed25519	8
2.6	Создали GPG ключ	8
2.7	Настроили и скопировали его	9
2.8	GPG ключ	9
2.9	Параметры репозитория	9
2.10	Привязка репозитория к аккаунту	10
2.11	Загрузка шаблона	10
2.12	Коммит	10

Список таблиц

1 Цель работы

Целью данной работы является изучение идеологии и применения средств контроля версий, освоение умения по работе с git.

2 Выполнение лабораторной работы

1. Установили git и gh.

```
mioannis@fedora:~$ git
использование: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
                [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
                [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
                [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
                [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
                <command> [<args>]

Стандартные команды Git используемые в различных ситуациях:

создание рабочей области (смотрите также: git help tutorial)
  clone      Клонирование репозитория в новый каталог
  init       Создание пустого репозитория Git или переинициализация существующего

работа с текущими изменениями (смотрите также: git help everyday)
  add        Добавление содержимого файла в индекс
  mv         Перемещение или переименование файла, каталога или символической ссылки
  restore    Восстановление файлов в рабочем каталоге
  rm         Удаление файлов из рабочего каталога и индекса

просмотр истории и текущего состояния (смотрите также: git help revisions)
  bisect     Выполнение двоичного поиска коммита, который вносит ошибку
  diff       Вывод разницы между коммитами, коммитом и рабочим каталогом и т.д.
  grep       Вывод строк, соответствующих шаблону
  log        Вывод истории коммитов
  show       Вывод различных типов объектов
  status     Вывод состояния рабочего каталога

выращивание, маркировка и правка вашей общей истории
  backfill   Download missing objects in a partial clone
  branch     Вывод списка, создание или удаление веток
  commit     Запись изменений в репозиторий
  history    EXPERIMENTAL: Rewrite history
  merge      Объединение одной или нескольких историй разработки вместе
  rebase     Повторное применение коммитов над верхушкой другой ветки
  reset      Set 'HEAD' or the index to a known state
  switch     Переключение веток
  tag        Create, list, delete or verify tags

совместная работа (смотрите также: git help workflows)
  fetch      Загрузка объектов и ссылок из другого репозитория
  pull       Извлечение изменений и объединение с другим репозиторием или локальной веткой
  push       Обновление внешних ссылок и связанных объектов

«git help -a» и «git help -g» выводит список доступных подкоманд
и небольшую справку по понятиям. Смотрите «git help <понятие>»
или «git help <термин>» чтобы узнать больше о конкретной подкоманде
или понятии.
Смотрите «git help git» для получения общего обзора системы.
mioannis@fedora:~$
```

Рисунок 2.1: Вывод команды git

2. Создаём учётную запись Github.

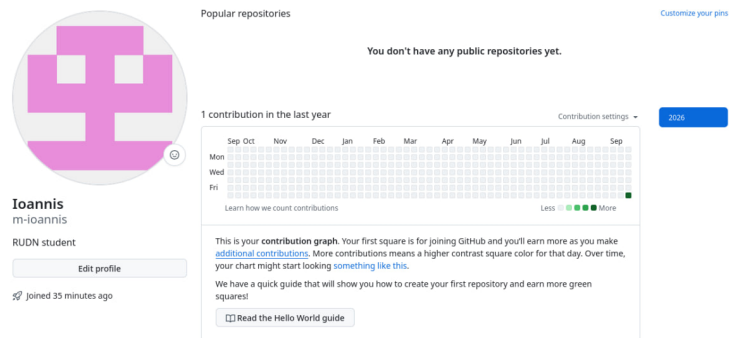


Рисунок 2.2: Мая новая учётная запись

3. Зададим почту и имя владельца репозитория, кодировку и другие данные.

```
root@fedora:~# git config --global user.name "Ioannis Magos"
root@fedora:~# git config --global user.email "1032240004@rudn.ru"
root@fedora:~# git config --global core.quotepath false
root@fedora:~# git config --global init.defaultBranch master
root@fedora:~# git config --global core.autocrlf input
root@fedora:~# git config --global core.safecrlf warn
root@fedora:~#
```

Рисунок 2.3: Параметры репозитория

4. Создаем SSH ключи

```
root@fedora:~# ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase for "/root/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:VBQHYEquONnHhjt1b0Ry20Z5Eeh8ri80FRfzD3yYIUA root@fedora
The key's randomart image is:
+---[RSA 4096]-----+
| .E.*o*o. |
| o + B = |
| * + * . |
| + % + . + |
| + * ^ S . |
| . & + . |
| + o o |
| o.o |
| .oo. |
+---[SHA256]-----+
root@fedora:~#
```

Рисунок 2.4: rsa-4096

```

root@fedora:~# ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519):
Enter passphrase for "/root/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519
Your public key has been saved in /root/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:Jfij+gwb7ETv2p+0Qe/79VYVufpse8ffLRXkAAwkPos root@fedora
The key's randomart image is:
+--[ED25519 256]--+
|      ..00.....|
|      ... . 0+ |
|      .0. . .+|
|      ..00    .0|
|      .E..S    .00|
|      0 0 0 .  .. =|
|      = + . . 0+.|
|      0 0 + . . .++|
|      =0*0*0. .0.+|
+-----[SHA256]-----+

```

Рисунок 2.5: ed25519

5. Создаем GPG ключ и копируем его в буфер обмена

```

root@fedora:~# gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/root/.gnupg'
Выберите тип ключа:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
  0 = не ограничен
  <n> = срок действия ключа - n дней
  <n>w = срок действия ключа - n недель
  <n>m = срок действия ключа - n месяцев
  <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: m-ioannis
Адрес электронной почты: 1032240004@rudn.ru
Примечание:
Вы выбрали следующий идентификатор пользователя:
  "m-ioannis <1032240004@rudn.ru>"

```

Рисунок 2.6: Создали GPG ключ

```

root@fedora:~# gpg --list-secret-keys --keyid-format LONG
gpg: проверка таблицы доверия
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: глубина: 0 достоверных: 1 подписанных: 0 доверие: 0-, 0q, 0n, 0m, 0f, 1u
[keybox]
-----
sec   rsa4096/3F0AA5758D79875C 2026-09-26 [sc]
      E54256E34F4DDA4ACD3AFE43F0AA5758D79875C
uid     [ абсолютно ] m-ioannis <103224004@rudn.ru>
ssb     rsa4096/F2668198080DC92C 2026-09-26 [E]

root@fedora:~# ^C
root@fedora:~# gpg --armor --export 3F0AA5758D79875C | xclip -sel clip
bash: xclip: команда не найдена...
Установить пакет «xclip», предоставляющий команду «xclip»? [N/y] y

* Ожидание в очереди...
Следующие пакеты должны быть установлены:
xclip_0.13-26.git11c8a81.fc44.x86_64 Command line clipboard grabber
Продолжить с этими изменениями? [N/y] y

* Ожидание в очереди...
* Ожидание аутентификации...
* Ожидание в очереди...
* Загрузка пакетов...
* Выполнение...
* Установка пакетов...

root@fedora:~# gpg --armor --export 3F0AA5758D79875C | xclip -sel clip
root@fedora:~#

```

Рисунок 2.7: Настроили и скопировали его

6. Добавляем GPG ключ в аккаунт

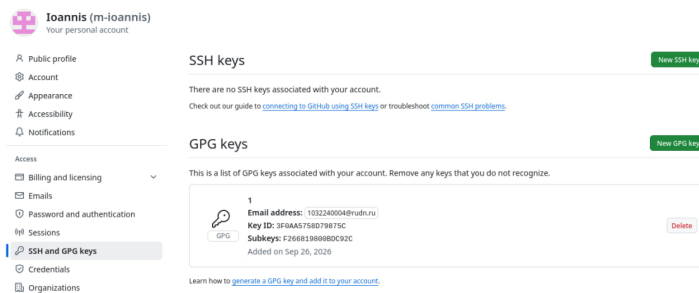


Рисунок 2.8: GPG ключ

7. Настройка автоматических подписей коммитов

```

root@fedora:~# gpg --armor --export 3F0AA5758D79875C | xclip -sel clip
root@fedora:~# git config --global user.signingkey 3F0AA5758D79875C
root@fedora:~# git config --global commit.gpgsign true
root@fedora:~# git config --global gpg.program $(which gpg2)
root@fedora:~#

```

Рисунок 2.9: Параметры репозитория

8. Настройка gh

```

root@fedora:~# gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /root/.ssh/id_rsa.pub
? Title for your SSH key: github-cli
? How would you like to authenticate GitHub CLI? Login with a web browser

| First copy your one-time code: 8F3D-5088
Press Enter to open https://github.com/login/device in your browser...
Running Firefox as root in a regular user's session is not supported. ($XAUTHORITY is /run/user/1000/.xauth-xylandauth.AK37V3 which is owned by mioannis.)

Authentication complete.
- gh config set -h github.com git.protocol ssh
- Configured git protocol
- Authentication credentials saved in plain text
- Uploaded the SSH key to your GitHub account: /root/.ssh/id_rsa.pub
- Logged in as m-ioannis
root@fedora:~#
root@fedora:~#

```

Рисунок 2.10: Привязка репозитория к аккаунту

9. Загрузка шаблона репозитория и синхронизация

```

root@fedora:~/work/study/2025-2026/Операционные системы/study_2026_os-intro# rm package.json
rm: удалить обычный файл 'package.json'?
root@fedora:~/work/study/2025-2026/Операционные системы/study_2026_os-intro# echo os-intro > COURSE
root@fedora:~/work/study/2025-2026/Операционные системы/study_2026_os-intro# make prepare
Synchronizing submodule url for 'template/report'
Synchronizing submodule url for 'template/presentation'
Synchronizing submodule url for 'template/presentation'
Synchronizing submodule url for 'template/report'

```

Рисунок 2.11: Загрузка шаблона

10. Подготовка репозитория и коммит изменений

```

root@fedora:~/work/study/2025-2026/Операционные системы/study_2026_os-intro# git add .
root@fedora:~/work/study/2025-2026/Операционные системы/study_2026_os-intro# git commit -am 'feat(main): make course structure'
root@fedora:~/work/study/2025-2026/Операционные системы/study_2026_os-intro# git status
Текущая ветка: master
Эта ветка соответствует «origin/master».
нечего коммитить, нет изменений в рабочем каталоге
root@fedora:~/work/study/2025-2026/Операционные системы/study_2026_os-intro#

```

Рисунок 2.12: Коммит

3 Вывод

Мы приобрели практические навыки работы с системой контроля версий git и GitHub, изучили идеологию и применение средств контроля версий.

4 Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий (Version Control System, VCS) используются при совместной работе нескольких человек над одним проектом. Как правило, основная структура проекта хранится в локальном или удалённом репозитории, доступ к которому предоставляется участникам проекта. При внесении изменений в проект система контроля версий позволяет сохранять и отслеживать их, объединять изменения, выполненные разными участниками, а также при необходимости возвращаться к любой предыдущей версии проекта.

2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.
 - **Хранилище** — пространство на накопителе, в котором находится репозиторий проекта.
 - **Commit** — фиксация текущего состояния хранилища в определённый момент времени.
 - **История** — последовательность изменений хранилища, представленная в виде списка выполненных коммитов.
 - **Рабочая копия** — локальная версия сетевого репозитория, с которой непосредственно работает программист. Она содержит текущее состоя-

ние файлов проекта, основанное на версии, загруженной из хранилища (обычно последней).

3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.

Централизованные системы контроля версий представляют собой приложения по типу клиент-сервер, в которых репозиторий проекта существует в единственном экземпляре и хранится на сервере. Доступ к нему осуществляется с помощью специального клиентского приложения. Примерами таких систем являются CVS и Subversion.

Распределенные системы контроля версий (Distributed Version Control System, DVCS) позволяют хранить копию репозитория у каждого разработчика, работающего с данной системой. При этом можно условно выделить центральный репозиторий, в который отправляются изменения из локальных репозиториях и с которым они синхронизируются. При работе с такой системой пользователи периодически синхронизируют свои локальные репозитории с центральным и работают непосредственно со своей локальной копией. После внесения достаточного количества изменений они отправляются на сервер. При этом сервер чаще всего является условным понятием, так как в большинстве DVCS отсутствует выделенный сервер с центральным репозиторием.

4. Опишите действия с VCS при единоличной работе с хранилищем.

Один пользователь работает над проектом и по мере необходимости выполняет коммиты, фиксируя отдельные этапы работы.

5. Опишите порядок работы с общим хранилищем VCS.

Несколько пользователей работают каждый над своей частью проекта. При этом каждый разработчик должен работать в отдельной ветке. После завершения работы ветка пользователя объединяется с основной веткой проекта.

6. Каковы основные задачи, решаемые инструментальным средством git?

- Ведение истории версий проекта: журнал (log), метки (tags), ветвления (branches).
- Работа с изменениями: их выявление (diff), слияние (patch, merge).
- Обеспечение совместной работы: получение версии с сервера и загрузка обновлений на сервер.

7. Назовите и дайте краткую характеристику командам git.

- git config - установка параметров
- git status - полный список изменений файлов, ожидающих коммита
- git add . - сделать все измененные файлы готовыми для коммита.
- git commit -m «[descriptive message]» - записать изменения с заданным сообщением.
- git branch - список всех локальных веток в текущей директории.
- git checkout [branch-name] - переключиться на указанную ветку и обновить рабочую директорию.
- git merge [branch] — соединить изменения в текущей ветке с изменениями из заданной.
- git push - запустить текущую ветку в удаленную ветку.
- git pull - загрузить историю и изменения удаленной ветки и произвести слияние с текущей веткой.

8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

- git remote add [имя] [url] — добавляет удалённый репозиторий с заданным именем;
- git remote remove [имя] — удаляет удалённый репозиторий с заданным именем;

- `git remote rename [старое имя] [новое имя]` — переименовывает удалённый репозиторий;
- `git remote set-url [имя] [url]` — присваивает репозиторию с именем новый адрес;
- `git remote show [имя]` — показывает информацию о репозитории.

9. Что такое и зачем могут быть нужны ветви (branches)?

Ветвление — это возможность работать с разными версиями проекта: вместо единой последовательности упорядоченных коммитов история может разделяться в определённых точках. Каждая ветвь содержит лёгковесный указатель HEAD на последний коммит, что позволяет создавать большое количество веток без значительных затрат. Ветка по умолчанию называется `master`, однако её рекомендуется называть в соответствии с функциональностью, которая в ней разрабатывается.

10. Как и зачем можно игнорировать некоторые файлы при commit?

Некоторые файлы можно исключить из коммитов с помощью файла `.gitignore`. Это нужно для того, что мы не хотим оставлять в коммите, например временные файлы, логи или настройки среды разработки.